

ML LAB EXPT – 8

Develop a program to demonstrate the working of the decision tree algorithm. Use Breast Cancer Data set for building the decision tree and apply this knowledge to classify a new sample.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
import matplotlib.pyplot as plt

# Load the dataset
data = pd.read_csv('breast_cancer.csv')

# Data preprocessing
# Drop the 'id' column and any empty columns (like 'Unnamed: 32')
data = data.drop(['id', 'Unnamed: 32'], axis=1, errors='ignore')

# Convert diagnosis to binary values (M=1, B=0)
data['diagnosis'] = data['diagnosis'].map({'M': 1, 'B': 0})

# Split into features (X) and target (y)
X = data.drop('diagnosis', axis=1)
y = data['diagnosis']

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Create and train the decision tree classifier
dt_classifier = DecisionTreeClassifier(max_depth=3, random_state=42)
dt_classifier.fit(X_train, y_train)

# Evaluate the model
y_pred = dt_classifier.predict(X_test)

print("Model Evaluation:")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
# Display the decision tree rules
print("\nDecision Tree Rules:")
tree_rules = export_text(dt_classifier, feature_names=list(X.columns))
print(tree_rules)

# Visualize the decision tree
plt.figure(figsize=(20, 10))
plot_tree(dt_classifier, feature_names=X.columns, class_names=['Benign',
'Malignant'],filled=True, rounded=True, fontsize=10)
plt.title("Decision Tree for Breast Cancer Classification")
plt.show()

# Function to classify a new sample
def classify_new_sample(features):
    """
    Classify a new sample using the trained decision tree.
    features: A dictionary containing feature values for the new sample
    Returns: The predicted class (0 for Benign, 1 for Malignant) and
    probability
    """
    # Create a DataFrame with the new sample's features
    new_sample = pd.DataFrame([features])

    # Ensure we only keep the columns used in training
    available_cols = [col for col in X.columns if col in
                                                                new_sample.columns]
    new_sample = new_sample[available_cols]

    # Add any missing columns with default values (0)
    for col in X.columns:
        if col not in new_sample.columns:
            new_sample[col] = 0

    # Reorder columns to match training data
    new_sample = new_sample[X.columns]

    # Make prediction
    prediction = dt_classifier.predict(new_sample)
    proba = dt_classifier.predict_proba(new_sample)

    return prediction[0], proba[0]
```

```
# Example: Classify a new sample (using only the relevant features)
new_sample_features = {
    'radius_mean': 17.99,
    'texture_mean': 10.38,
    'perimeter_mean': 122.8,
    'area_mean': 1001,
    'smoothness_mean': 0.1184,
    'compactness_mean': 0.2776,
    'concavity_mean': 0.3001,
    'concave points_mean': 0.1471,
    'symmetry_mean': 0.2419,
    'fractal_dimension_mean': 0.07871,
    'radius_se': 1.095,
    'texture_se': 0.9053,
    'perimeter_se': 8.589,
    'area_se': 153.4,
    'smoothness_se': 0.006399,
    'compactness_se': 0.04904,
    'concavity_se': 0.05373,
    'concave points_se': 0.01587,
    'symmetry_se': 0.03003,
    'fractal_dimension_se': 0.006193,
    'radius_worst': 25.38,
    'texture_worst': 17.33,
    'perimeter_worst': 184.6,
    'area_worst': 2019,
    'smoothness_worst': 0.1622,
    'compactness_worst': 0.6656,
    'concavity_worst': 0.7119,
    'concave points_worst': 0.2654,
    'symmetry_worst': 0.4601,
    'fractal_dimension_worst': 0.1189
}

prediction, probability = classify_new_sample(new_sample_features)
print("\nNew Sample Classification:")
print(f"Predicted class: {'Malignant' if prediction == 1 else 'Benign'}")
print(f"Probability: Benign={probability[0]:.2f},
      Malignant={probability[1]:.2f}")
```

Output:

Model Evaluation:
Accuracy: 0.96

Classification Report:

	precision	recall	f1-score	support
0	0.96	0.98	0.97	108
1	0.97	0.94	0.95	63
accuracy			0.96	171
macro avg	0.97	0.96	0.96	171
weighted avg	0.96	0.96	0.96	171

Confusion Matrix:

```
[[106  2]
 [ 4 59]]
```

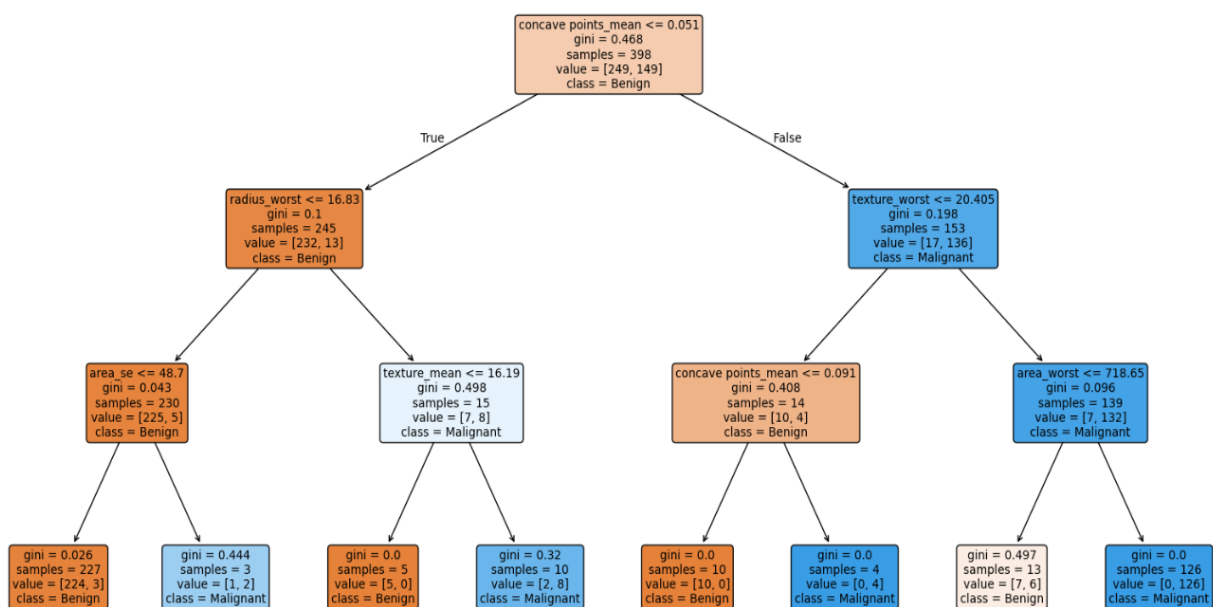
Decision Tree Rules:

```

|--- concave points_mean <= 0.05
|   |--- radius_worst <= 16.83
|   |   |--- area_se <= 48.70
|   |   |   |--- class: 0
|   |   |--- area_se > 48.70
|   |   |   |--- class: 1
|   |--- radius_worst > 16.83
|   |   |--- texture_mean <= 16.19
|   |   |   |--- class: 0
|   |   |--- texture_mean > 16.19
|   |   |   |--- class: 1
|--- concave points_mean > 0.05
|   |--- texture_worst <= 20.40
|   |   |--- concave points_mean <= 0.09
|   |   |   |--- class: 0
|   |   |--- concave points_mean > 0.09
|   |   |   |--- class: 1
|   |--- texture_worst > 20.40
|   |   |--- area_worst <= 718.65
|   |   |   |--- class: 0
|   |   |--- area_worst > 718.65
|   |   |   |--- class: 1

```

Decision Tree for Breast Cancer Classification



New Sample Classification:
Predicted class: Malignant
Probability: Benign=0.00, Malignant=1.00